

Ejercicio 2013 — Control de Fábrica

Sensores serie en tiempo real — resolución inciso por inciso (a–f)

Ingeniería de Software II · ITBA · 2026-1C · Parcial resuelto en clase

RESUMEN

Caso canónico de la **Clase 4**: instrumentar una planta industrial con sensores por puerto serie que sufren *hangs* transitorios. La clave es que **Reliability** es el atributo #1 — de ahí Pollers redundantes con heartbeat (sin SPOF), Producer-Consumer para absorber picos de sensado, y separación Daemon (24/7, sin UI) / Webapp (sesiones humanas). Se resuelve con el método ADD (source: ejercicio-2013-control-fabrica.md).

§0 Enunciado y dominio

Una empresa industrial debe instrumentar su planta con **sensores conectados por puerto serie** (RS-232/RS-485). El sistema debe leer, agregar, presentar y archivar mediciones, con alta disponibilidad y detección temprana de anomalías (source: raw/exercises/Ejercicio 2013.pdf).

CUÁNDO NO · CUIDADO

Hecho del dominio, no excepción: el protocolo serie real tiene **latencias variables** y tiende a *hangs* transitorios. Parada de planta = pérdida económica significativa. Diseñar como si los sensores fueran confiables es el error que mata la respuesta.

Sensores en campo

RS-232/485; sistema externo, sin control sobre su firmware.

Planta productiva

Depende del sistema para operar; downtime es caro.

Operadores

Monitorean métricas en tiempo real.

Ingeniería

Analiza series históricas para mejorar procesos.

Compliance

Puede exigir trazabilidad de mediciones.

§a Stakeholders y atributos de calidad

Paso 1-2 del *Attribute Driven Design* ([Attribute Driven Design]): identificar stakeholders y, por cada uno, qué atributos les importan. **No saltar al dibujo de cajas** antes de articular esto.

TABLA 1. Stakeholders → atributos de calidad

Stakeholder	Atributos de calidad
Operadores de planta	Availability, Usability, Performance (tiempo real)
Ingeniería de proceso	Reliability de datos, Precision, Auditability
IT / Mantenimiento	Manageability, Supportability, Portability (a nuevos sensores)
Dirección	Costos, time-to-market
Compliance	Auditability, Security (integridad de lecturas)

Olvidar **Auditability** es un killer en industria regulada (source: ejercicio-2013-control-fabrica.md).

Selección priorizada (pizarrón, Clase 4)

De los muchos atributos posibles, el profesor priorizó en el pizarrón cuatro y en este orden (source: raw/classes/2026-04-09 - Clase 4/clase4.pdf):

1 · RELIABILITY	2 · PRECISION	3 · PERFORMANCE	4 · INTEROPERABILITY
-----------------	---------------	-----------------	----------------------

Reliability

Sistema con ACL, control de acceso. Es el atributo dominante — de él se deriva la redundancia de pollers.

Precision

Errores tipo 1 y tipo 2 en la detección de anomalías (falsos positivos / falsos negativos).

Performance

Velocidad de sensado.

Interoperability

Soportar cambios de hardware y sensores nuevos.

§b b Árbol de utilidad

DEFINICIÓN

Árbol de utilidad: raíz "utilidad" → atributos dominantes → features → **escenarios concretos hoja**, cada uno etiquetado (importancia, dificultad) con escala H/M/L ([[Árbol de utilidad]]). Es el producto del paso ADD.

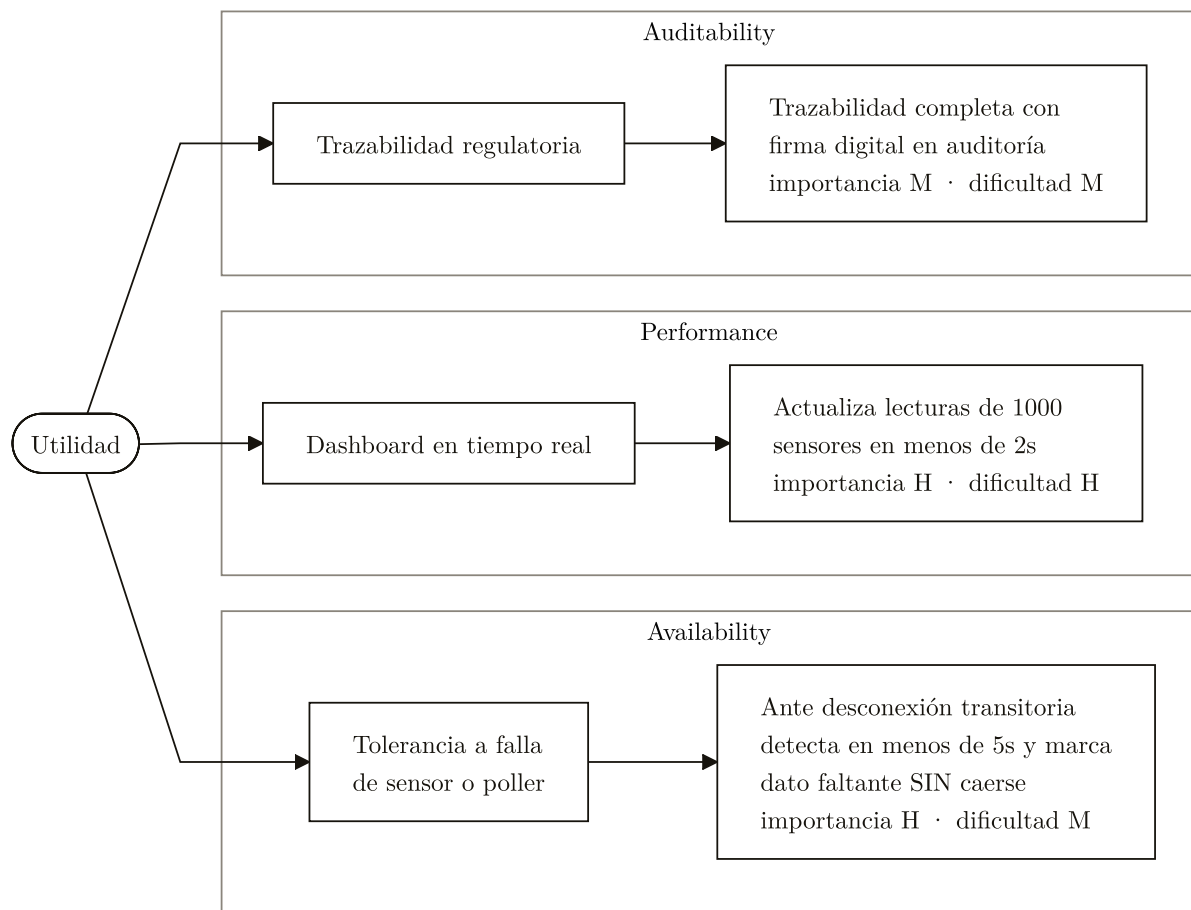


FIGURA 1. Árbol de utilidad ADD: raíz *Utilidad* → atributos dominantes → feature → escenario hoja cuantificado con su par importancia/dificultad. El escenario H,H justifica la arquitectura.

REGLA DE ORO

Un escenario hoja bien formado tiene **estímulo + respuesta medible** (" $<5s$ ", " $<2s$ "). Los escenarios operacionales del árbol son SLOs potenciales ([[SLA, SLO, SLI]]). Los (H,H) son los que justifican la arquitectura.

§c c Arquitectura propuesta (desarrollo de pizarra)

Arquitectura candidata desarrollada en la [[Clase 4 — ¿Cuándo diseñamos?]] (source: raw/clases/2026-04-09 - Clase 4/clase4.pdf):

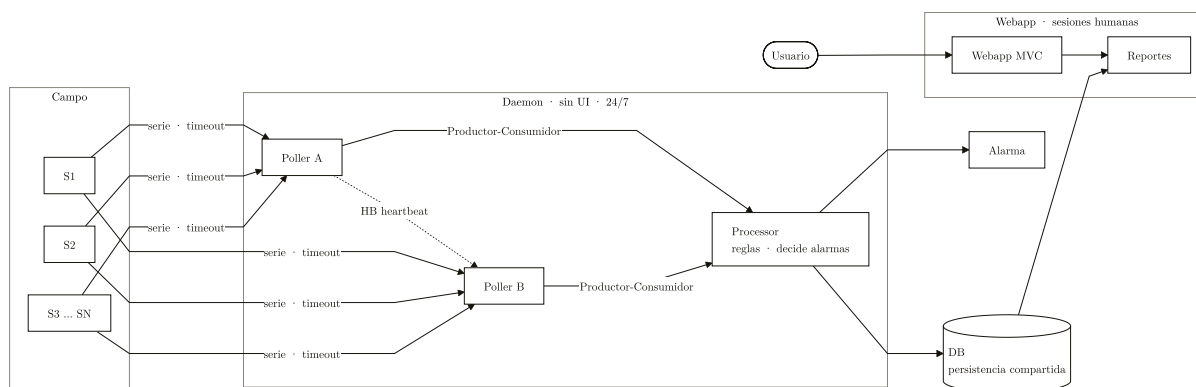


FIGURA 2. *Arquitectura de componentes: Daemon sin UI orquesta pollers redundantes con heartbeat sobre los sensores serie, alimenta al Processor vía Producer-Consumer, que persiste en DB y dispara Alarma; la Webapp MVC lee reportes de la DB.*

TABLA 2. Componentes y responsabilidades

Componente	Responsabilidad
Sensores S1...SN	Lectura física en campo. Sistema externo, sin control sobre su firmware.
Pollers (≥2, redundantes)	Lectura periódica de sensores. Implementan timeout y heartbeat entre sí para detectar caída del par y tomar el control sin downtime.
Daemon (sin UI)	Contenedor que orquesta los pollers. Sin interfaz humana — sólo procesamiento.
Processor	Recibe datos del daemon (vía Producer-Consumer), aplica reglas, decide alarmas, persiste en DB.
Alarma	Output del processor ante condición crítica.
DB	Persistencia compartida: processor escribe, reportes lee.
Webapp (MVC)	Interfaz humana: dashboards y reportes históricos ([[MVC]]).

Notas de pizarra

- *Poller* → *timeout local* → *publica* → *error*.
- *2do timeout* → *SLO* + *alerta*.
- *Processor* → *guarda datos en tabla/central* → *publica*.
- Mecanismos auxiliares anotados: **message schema** → **TTL**, **cluster**, **redirect order**, queues con **FIFO**, **circuit breaker** (fallback solid/abierto), pre-warm de caches volátiles (source: raw/classes/2026-04-09 - Clase 4/clase4.pdf).

Justificación de las tres decisiones clave

Pollers redundantes + heartbeat: Reliability es el atributo #1. Un único Poller es **SPOF**; dos con heartbeat resuelven la falla del Poller sin perder muestras. Costo: coordinación (¿quién toma el lock con ambos vivos?), típicamente leader election o lock distribuido.

Daemon sin UI separado del Webapp: distinto perfil operativo. El daemon corre 24/7 atendiendo sensores; el webapp atiende sesiones humanas. Separarlos permite escalar y desplegar independientemente — separation of concerns materializado en proceso.

Producer-Consumer Pollers → Processor: desacopla la velocidad de sensado (variable, *bursty*) de la de procesamiento (uniforme, ACID a DB). La cola en el medio absorbe los picos.

Decisión equivalente en lenguaje de estilos

Generalizado a [[Estilos arquitectónicos]]: [[**Pipes and Filters**]] + [[**Publish-Subscribe**]] con persistencia histórica.

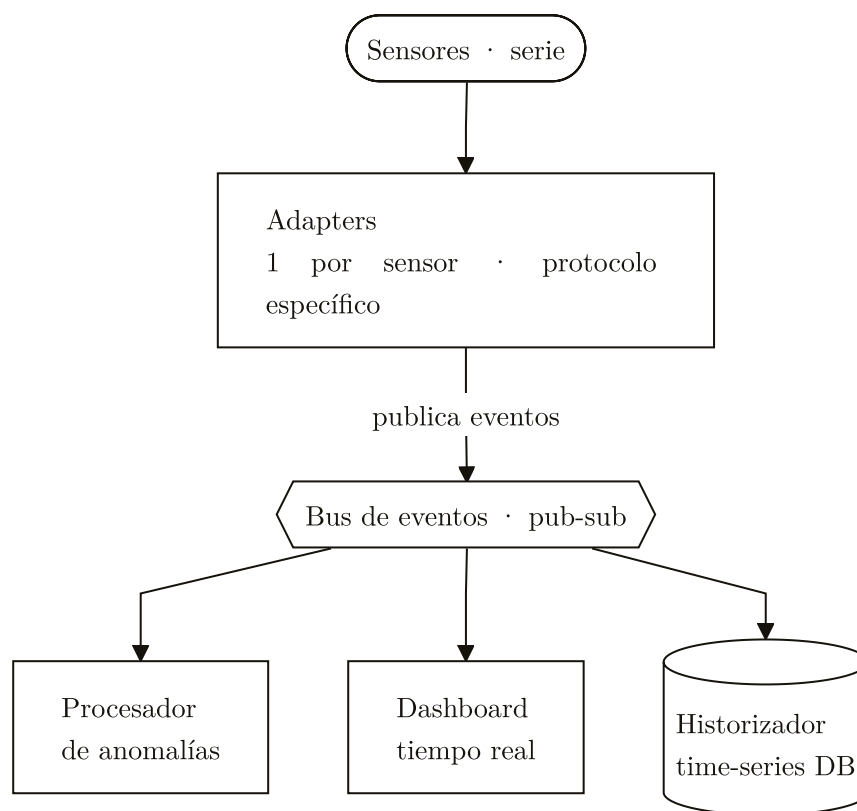


FIGURA 3. Misma decisión en lenguaje de estilos: Pipes and Filters + Publish-Subscribe. Adapters por protocolo aíslan cada sensor y publican al bus; los consumidores se suscriben de forma independiente con persistencia histórica en time-series DB.

CUÁNDO SÍ · VENTAJAS

- **Pub-sub** desacopla adapters de consumidores: nuevo consumidor = nueva suscripción, sin tocar productores.
- **Adapters por protocolo** aíslan las particularidades de cada sensor → mejora portability/interoperability.
- **Time-series DB** (InfluxDB, TimescaleDB) optimizada para el patrón escritura/consulta de mediciones.

§d d Atributos y trade-offs

TABLA 3. Trade-offs explícitos

Tensión	Gana	Resigna / costo
Redundancia de bus / pollers	Availability	Simplicity (complejidad de coordinación)
Firmar cada lectura	Auditability	Performance (throughput)
Adapters con interfaz común	Portability	Complexity de diseño

TRADE-OFF

Performance ↔ Auditability: firmar cada evento mata el throughput. Se resuelve firmando lotes en vez de cada lectura individual — se conserva trazabilidad sin pagar la firma por muestra (source: ejercicio-2013-control-fabrica.md).

TRADE-OFF

Availability ↔ Simplicity: dos pollers con heartbeat eliminan el SPOF pero introducen el problema del lock/leader election. La complejidad es el precio de no perder muestras ante caída de un poller.

§e e Riesgos y mitigaciones

TABLA 4. Registro de riesgos

Riesgo	Mitigación
Sensor defectuoso envía datos espurios	Validación + anomaly detection en el processor
Caída del bus	Bus replicado (Kafka ≥ 3 brokers); adapters con buffer local
Crecimiento del dataset histórico	Retention policies + downsampling automático
Nuevo protocolo futuro	Adapter pluggable con interfaz estable
Pérdida de un operador crítico	Runbooks + dashboards auto-explicativos

CUÁNDO NO • CUIDADO

Anti-patrones que tumban la resolución:

- "Guardo todo en una BD relacional grande" — no escala para time-series ni soporta consultas real-time.
- "Cada sensor habla directo con el dashboard" — acopla fuerte; difícil agregar consumidores.
- "Monolito con un thread por sensor" — frágil, no escala horizontalmente.
- Olvidar Auditability — killer en industria regulada.
- No considerar el serie real — los hangs transitorios son un hecho, no una excepción.

§f f Evaluación y siguiente iteración

Aplicar [[**Lightweight ATAM**]] al diseño antes de construir: identificar *sensitivity points* y trade-off points.

- **Sensitivity point:** el bus pub-sub como SPOF potencial.
- **Sensitivity point:** el formato de mensaje como contrato crítico (cambiarlo rompe a todos los consumidores).
- **Trade-off point:** firma de lecturas (Performance vs Auditability, ver §d).

FUENTE

Los pasos (a) y (b) son literalmente ADD 1-2; (c) elige estilo; (f) lo evalúa con ATAM.
Bibliografía: Hohpe & Woolf, Enterprise Integration Patterns (pub-sub); Kleppmann, Designing Data-Intensive Applications (time-series, event logs); Bass, Clements, Kazman, Software Architecture in Practice (source: ejercicio-2013-control-fabrica.md).

EN EL PARCIAL

Respuesta mínima aprobada: stakeholders + atributos priorizados con justificación + árbol de utilidad con escenarios **cuantificados** + diagrama de componentes nombrados + ≥2 trade-offs explícitos + riesgos. El diferencial de este caso: nombrar heartbeat/redundancia de pollers y Producer-Consumer, y atarlos a Reliability como atributo #1.

Pregunta a profundizar: ¿cómo cambia el diseño si los sensores migran a IoT inalámbrico (MQTT, LoRaWAN)? ¿Qué atributos se vuelven dominantes — seguridad, battery life, cobertura?